

MHTML

MEGALITHIC HTML FORMAT

Technical Specification

Volume 1: Foundation Architecture, PWA Orchestration, & Single-File Paradigms

Specification Version	2026.1.0
Document Status	Production Reference
Artifact Format	.html (Megalithic) — runs seamlessly in any modern browser
Target Audience	Architects, Full-Stack Engineers, Offline-First Developers
Release Date	August 2026

TABLE OF CONTENTS

1. Architectural Philosophy & The Megalithic Paradigm 2

2. High-Density Single-File Encapsulation Framework 3

3. PWA Engine: Self-Registration via Virtual Blob Compilation 4

4. Advanced In-Line Scoped Templating & Reactivity 5

5. Multi-Layer Versioning Ledger & Git Interoperability 6

6. System Diagnostics and Fail-Safe Error Journals 7

7. Advanced SEO, Metadata & Discoverability Layer 8

8. Strategic Deployment Use Cases & Artifact Runtime 9

9. Framework Reference Implementation Blueprint 10

10. Extension Roadmap & Future Modules 10

1. Architectural Philosophy & The Megalithic Paradigm

The modern web ecosystem is heavily fractured by build steps, multi-directory deployment pipelines, complex dependency graphs, and hydration-heavy frameworks. While these systems serve distributed enterprise infrastructure, they introduce immense friction for greenfield projects, architectural spikes, offline-first utilities, and isolated trial applications.

MHTML (Megalithic HTML) is an alternative architectural pattern designed to collapse the entire application lifecycle—structure, styling, reactivity, client-side data state, orchestration, version history, compilation logs, Progressive Web App configuration, and advanced SEO metadata—into a single, deployable, human-readable `.html` file typically ranging from 5,000 to 50,000+ lines of code.

The Seven-Layer Megalith Stack

Layer	Name	Responsibility
1	Embedded Engine System	Custom Bootloader & Dependency Registry
2	Compiler & Template Engine	Reactive Virtual DOM & In-Line Scopes
3	PWA Orchestration Layer	Base64 / Blob Service Worker Engine & Cache
4	State Management Network	Immutable Store & Schema Validation
5	Component Matrix	Scoped CSS, HTML Shells, Component JS
6	Metadata & Version Ledger	Cryptographic Signatures, SemVer Logs, SEO
7	System Diagnostic Journal	High-Fidelity Crash & Telemetry Logs

Megalithic design treats the single HTML document not as a simple webpage, but as an **encapsulated virtual operating system disk**. By consolidating the execution context, MHTML eliminates network latency variations between asset chunks, eradicates build-environment synchronization errors, and ensures that the entire application can be downloaded, shared via email, executed offline from a local hard drive, or served instantly from a bare-metal content delivery network (CDN).

Core Design Principle — Artifact Format

An MHTML file is a first-class **artifact**: a self-describing, self-bootstrapping, installable package that runs seamlessly in any standards-compliant browser without external dependencies, build steps, or network requirements after the initial load. It is simultaneously source code, binary, and runtime.

Why Single-File Wins for Specific Classes of Software

- **Zero configuration surface** — no `package.json`, no webpack, no Vite, no lockfiles.
- **Atomic versioning** — one hash represents the entire application state.
- **Air-gapped readiness** — works on machines with no internet after download.
- **Instant shareability** — email, USB, chat, or drop into any static host.
- **Deterministic boot** — identical behavior regardless of CDN edge location or cache headers.

2. High-Density Single-File Encapsulation Framework

To prevent a 10,000–50,000-line file from degrading into unmaintainable code, MHTML enforces a strict structural anatomy using specialized `id` and `data-*` attributes on HTML `<script>`, `<template>`, and `<style>` blocks. Every system concern lives in a named, queryable block.

Canonical Document Skeleton

```
<!DOCTYPE html>
<html lang="en" data-mhtml-version="1.0.0" data-mhtml-spec="2026.1.0">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <!-- SECTION 1: LEDGER + SEO + MANIFEST METADATA -->
  <script id="mhtml-ledger" type="application/json"
    data-system-block="ledger">...</script>
  <!-- SECTION 2: GLOBAL + COMPONENT STYLES -->
  <style id="mhtml-global-styles" data-system-block="style-global">...</style>
</head>
<body>
  <div id="mhtml-runtime-root"><!-- Hydration target --></div>
  <!-- SECTION 3: COMPONENT MATRIX (templates + scoped styles) -->
  <template id="tpl-dashboard" data-component="dashboard">...</template>
  <style id="style-dashboard" data-scope-target="dashboard">...</style>
  <!-- SECTION 4: CORE ENGINE -->
  <script id="mhtml-core-engine" data-system-block="engine">...</script>
  <!-- SECTION 5: PWA SERVICE WORKER (plain text) -->
  <script id="mhtml-serviceworker-engine" type="text/plain"
    data-system-block="pwa-worker">...</script>
  <!-- SECTION 6: DIAGNOSTIC LEDGER -->
  <script id="mhtml-diagnostic-ledger" type="application/json"
    data-system-block="diagnostics">[]</script>
</body>
</html>
```

Block Identity Contract

Every system block must carry a unique `id` and a `data-system-block` (or `data-component` / `data-scope-target`) attribute. The core engine discovers blocks exclusively via these selectors. Renaming or omitting them breaks the boot sequence by design — this is intentional guardrail behavior that prevents silent partial initialization.

Ledger Schema (excerpt)

```
{
  "specVersion": "2026.1.0",
  "applicationId": "org.megalith.matrix.control",
  "versionHistory": [
    { "version": "1.0.0", "timestamp": "2026-08-06T03:15:00Z",
      "author": "System Architect", "hash": "sha256-e3b0c442..." }
  ],
  "dependencies": {
    "engine": "internal://mhtml-core-engine",
    "ui-library": "embedded://tailwind-3.4.1"
  },
  "seo": { "title": "...", "description": "...", "canonical": "..." }
}
```

Maintainability Rule

Never interleave component logic inside the core engine block. Keep templates, scoped styles, and component scripts in their own named regions. The human-readable section comments (SECTION 1...6) are part of the specification and must be preserved for tooling and code review.

3. PWA Engine: Self-Registration via Virtual Blob Compilation

A standard Progressive Web App requires separate assets: `manifest.json` and a standalone `sw.js` file. Within the MHTML definition, external network files are strictly prohibited. The system generates its own network ecosystem entirely in-memory at runtime.

The Virtual Lifecycle Matrix

1. Core Bootloader Sequence — Upon initial script execution the MHTML bootloader extracts the string contents of `<script id="mhtml-serviceworker-engine">`.

2. Blob Generation Engine — The core turns this plain-text code block into an immutable application package:

```
const swBlob = new Blob([rawTextCodeString], { type: 'application/javascript' });
const virtualBlobURL = URL.createObjectURL(swBlob);
navigator.serviceWorker.register(virtualBlobURL);
```

3. Dynamic Manifest Ingestion — A Web App Manifest cannot be registered from a text Blob URL directly via static meta links. MHTML overcomes this constraint by dynamically generating a Blob URL and injecting a `<link rel="manifest">` tag at runtime:

```
const manifest = {
  name: 'MHTML Megalithic Control Matrix',
  short_name: 'MHTML',
  start_url: window.location.href,
  display: 'standalone',
  background_color: '#0f172a',
  theme_color: '#3b82f6',
  icons: [{ src: 'data:image/svg+xml;base64,...', sizes: '512x512',
    type: 'image/svg+xml' }]
};
const blob = new Blob([JSON.stringify(manifest)], {type:'application/json'});
const url = URL.createObjectURL(blob);
const link = document.createElement('link');
link.rel = 'manifest'; link.href = url;
document.head.appendChild(link);
```

By using data URIs and Object Blobs, an MHTML file successfully transforms itself into an installation-ready PWA package within milliseconds of hitting the client browser. The same technique is used for offline fallback: the service worker caches the single root document and serves it for any navigation request when the network is unavailable.

Security Note

Blob URLs are same-origin with the document that created them. Service-worker registration from a Blob is supported in all modern browsers that implement the Service Worker specification. Always call `URL.revokeObjectURL` after successful registration if the Blob is no longer needed, to free memory.

4. Advanced In-Line Scoped Templating & Reactivity

When an application file crosses 5,000 lines of code, vanilla JavaScript state mutation leads to immediate state decay and synchronization failures. MHTML features an embedded declarative UI hydration framework that reads `<template>` fragments and links them to scoped styles without breaking code readability.

Scoped Custom DOM Trees

The engine matches custom style blocks to templates using a `data-scope-target` selector query. During runtime parsing, the engine maps component nodes and updates elements without recalculating the layout rules of the entire page. Style selectors are rewritten so that `.btn` becomes `.btn[data-scope-css="componentName"]`:

```
function applyScopedStyles(componentName) {
  const raw = document.querySelector(
    `style[data-scope-target="${componentName}"]`);
  if (!raw) return;
  const transformed = raw.textContent.replace(
    /([\r\n,{}]+)(?=\s{}/g, (match) => {
      return match.split(',').map(sel => {
        const t = sel.trim();
        if (t === ':root' || t.startsWith('@media')) return t;
        return `${t}[data-scope-css="${componentName}"]`;
      }).join(', ');
    });
  const el = document.createElement('style');
  el.textContent = transformed;
  document.head.appendChild(el);
}
```

Reactive Data-Binding Loop

The reactive engine relies on an internal Proxy wrapper that intercepts mutations to the central state store, selectively updating DOM fields marked with `data-bind`:

```
const CreateStore = (initial, onUpdate) => new Proxy(initial, {
  set(target, prop, value) {
    target[prop] = value;
    onUpdate(prop, value);
    return true;
  }
});
// Usage: store.systemStatus = 'ONLINE';
// → all [data-bind="systemStatus"] nodes refresh
```

Event wiring uses declarative attributes such as `data-on-click="handlerName"`. The engine binds these at hydration time to methods registered on the component or global core instance, keeping markup free of inline JavaScript while remaining fully searchable and diffable.

5. Multi-Layer Versioning Ledger & Git Interoperability

Because an MHTML file functions as an entirely independent project container, tracking development changes directly inside the code ledger is critical. The structural metadata element `<script id="mhtml-ledger" type="application/json">` stores historical tags using Semantic Versioning rules.

Repository State Model (Logical View)

```
MHTML REPOSITORY STATE FILE (.html)
■■ HEAD Commit Pointer ■■■ [v1.0.4-production]
■■ Security Hash Validation Ledger (SHA-256 Signatures)
■■ Delta Audit Table:
    ■■■ v1.0.4 ■■ [2026-08-06] ■■ Fix PWA Cache Strategy
    ■■■ v1.0.3 ■■ [2026-07-22] ■■ Add Offline WebDB Engine
    ■■■ v1.0.0 ■■ [2026-06-01] ■■ Greenfield Base System Boot
```

Git Pre-Commit Integration

To sync smoothly with corporate Git pipelines, automated pre-commit shell scripts compute the SHA-256 integrity hash of the individual script blocks (engine, worker, ledger, each component), injecting the resulting hash array directly into the versionHistory configuration block before pushing updates. This lets deployment engines detect file manipulation without parsing the entire file structure.

```
# Example pre-commit fragment
ENGINE_HASH=$(sha256sum mhtml-core-engine.js | cut -d' ' -f1)
jq --arg h "$ENGINE_HASH" \
  '.versionHistory[0].blockHashes.engine = $h' \
  ledger.json > ledger.tmp && mv ledger.tmp ledger.json
# Then re-embed ledger into the single .html artifact
```

Because the entire application is one file, a single Git commit represents a complete, runnable snapshot. Bisecting becomes trivial: every intermediate commit is a fully functional artifact that can be opened in a browser without rebuild steps.

Integrity Guarantee

At boot the core engine recomputes hashes of critical blocks and compares them against the values stored in the ledger. A mismatch raises a `DIAGNOSTIC_INTEGRITY_FAILURE` entry and can optionally halt hydration in high-security deployments.

6. System Diagnostics and Fail-Safe Error Journals

If a single-file application crashes while running locally in an offline environment, finding the bug can be incredibly difficult without external server monitoring. MHTML builds recovery safety nets directly into the application file structure using the `<script id="mhtml-diagnostic-ledger">` container block.

High-Fidelity Crash Interception Pipeline

- 1. Uncaught Error Trap** — The engine configures global event handlers immediately upon script execution.
- 2. State Serialization** — The telemetry script serializes the current variables inside the state tracking system alongside stack trace arrays.
- 3. Internal Ledger Writing** — The system modifies its own DOM nodes in real-time. If a user crashes during an offline session, they can click a recovery button to download the current state of the document—including the freshly generated error logs—saving it as a debugging file for developers.

```
window.addEventListener('error', (e) => {
  MHTML.recordError('CRITICAL_EXCEPTION', e.message, e.stack);
});
window.addEventListener('unhandledrejection', (event) => {
  MHTML.recordError('UNHANDLED_PROMISE_REJECTION',
    event.reason?.message || String(event.reason),
    event.reason?.stack || '');
});
```

Diagnostic Entry Schema

```
{
  "timestamp": "2026-08-06T03:15:02.110Z",
  "level": "CRITICAL" | "ERROR" | "WARN" | "INFO" | "DEBUG",
  "message": "Human-readable summary",
  "stack": "Optional stack trace string",
  "context": { "component": "dashboard", "stateSnapshot": {...} }
}
```

The diagnostic ledger is itself a live, queryable JSON array inside the document. Export is performed by serializing the entire HTML document (or just the ledger node) via `Blob + URL.createObjectURL`, producing a self-contained debug artifact that can be attached to an issue tracker.

7. Advanced SEO, Metadata & Discoverability Layer

Although MHTML is designed for offline and artifact distribution, many deployments still require first-class search-engine and social-media discoverability when the file is served over HTTP. The specification therefore mandates a comprehensive, machine-readable metadata surface that lives inside the same single file.

Required SEO & Social Surface

Element	Purpose	MHTML Location
title + meta description	Primary SERP snippet	<head> + ledger.seo
canonical link	Duplicate-content control	<link rel=canonical>
Open Graph / Twitter cards	Social share previews	meta og:* / twitter:*
JSON-LD Structured Data	Rich results, SoftwareApplication	script type=application/ld+json
robots / noindex flags	Crawl policy for private artifacts	meta robots + ledger
theme-color / msapplication	PWA & browser chrome	meta + dynamic manifest

JSON-LD SoftwareApplication Example

```
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "MHTML Control Matrix",
  "applicationCategory": "BusinessApplication",
  "operatingSystem": "Any (Browser)",
  "offers": { "@type": "Offer", "price": "0" },
  "version": "1.0.0",
  "author": { "@type": "Organization", "name": "Megalith" }
}
```

All SEO values are also mirrored inside the mhtml-ledger JSON so that build tooling and the runtime itself can read a single source of truth. When the application is distributed as a pure offline artifact, the robots meta may be set to noindex to prevent accidental public indexing of internal tools.

8. Strategic Deployment Use Cases & Artifact Runtime

MHTML is not a replacement for every multi-page or micro-frontend architecture. It is the optimal choice for three well-defined vectors:

Application Vector A — One-Off Utility Systems

Ideal for internal infrastructure tasks, disaster-recovery consoles, field telemetry tools, and deep-field data gathering forms where complex cloud environments are unavailable or undesirable. A single HTML file can be carried on a USB stick and executed on air-gapped workstations.

Application Vector B — Greenfield Micro-Prototyping

Allows development teams to prototype full architectural ideas inside a single file, validating user paths without spending days configuring microfrontend pipelines, module bundlers, or hosting endpoints. Once validated, the same file can be progressively decomposed into a conventional multi-file codebase if scale demands it.

Application Vector C — Air-Gapped Client Deliverables

Enterprise software teams can distribute fully-functional trial products as a single .html file. Customers evaluate complex web apps offline within secure networks, completely eliminating cloud-configuration compliance friction and trial-environment setup cost.

Artifact Runtime Guarantees

- Opens and runs in any Chromium, Gecko, or WebKit browser (desktop & mobile).
- Installable as a PWA when served over HTTPS or localhost.
- Fully functional offline after first load (service-worker + cache).
- Shareable via email, chat, cloud drive, or physical media without modification.
- Hash of the file is the sole integrity and version identifier.
- No external network calls required for core operation (optional telemetry may be disabled).

Serving Recommendation

When hosting publicly, serve the file with Content-Type text/html and Cache-Control that respects the embedded version. For maximum cache efficiency the filename itself may include the SemVer (e.g., matrix-1.0.4.html). The service worker still treats the document as the sole cache entry.

9. Framework Reference Implementation Blueprint

The following production-oriented core sketch demonstrates component registration, reactive state, diagnostic logging, and hydration inside a unified megalithic runtime. Full source is expected to live inside the `mhtml-core-engine` block of a real artifact.

```
class MhtmlEngineCore {
  constructor() {
    this.systemLogRegistry = [];
    this.reactiveStateStore = null;
    this.registeredComponents = new Set();
  }
  initializeCoreSystem(schema) {
    this.setupGlobalErrorHandler();
    this.setupReactiveDataStore(schema);
    this.parseEmbeddedComponentMatrix();
  }
  setupReactiveDataStore(schema) {
    const self = this;
    this.reactiveStateStore = new Proxy(schema, {
      set(t, p, v) {
        t[p] = v;
        self.triggerDomSyncForProperty(p, v);
        return true;
      }
    });
  }
  triggerDomSyncForProperty(key, val) {
    document.querySelectorAll(`[data-bind="${key}"]`)
      .forEach(n => {
        if (n.tagName === 'INPUT' || n.tagName === 'TEXTAREA')
          n.value = val;
      });
  }
  ... (truncated for print) ...
}
```

10. Extension Roadmap & Future Modules

Volume 2 and subsequent releases will formalize the following modules that can already be prototyped inside the current single-file paradigm:

- **Automated Build Tooling** — CLI that concatenates modular source directories into a single validated MHTML artifact, injecting hashes and updating the ledger.
- **IndexedDB Sync Layers** — Declarative offline data schemas with conflict-resolution policies and optional background sync when connectivity returns.
- **Extended Base64 Asset Embedding** — Standardized routines for fonts, icon sets, and high-resolution images with size budgets and lazy decoding.
- **Component Hot-Swap Protocol** — Runtime replacement of template + style + logic blocks from another MHTML file for A/B experiments without full reload.
- **Cryptographic Signing** — Optional detached or embedded signatures so receiving parties can verify authenticity of the artifact before execution.

This concludes Volume 1 of the MHTML Technical Specification. The format is intentionally simple at the surface (one HTML file) yet deliberately rigorous in structure so that tooling, security review, and long-term maintenance remain tractable even as individual artifacts grow to tens of thousands of lines.

Megalithic HTML — One file. Entire system. Instant runtime.

Specification 2026.1.0 • Proprietary Artifact Format • Runs seamlessly in HTML